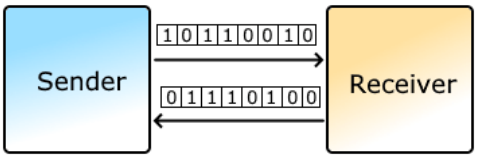
Lesson 4: Micro-controller Communication

Microcontroller Communication protocols are an important feature when building complex embedded systems. Some of the key reasons why communication is important include:

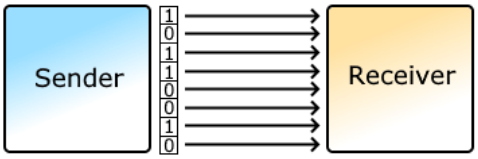
1. Communication allows for interfacing with different modules for input and output.
2. Communication expands the capabilities of embedded system by enable module communication
3. Communication cross platform interactions between different microcontroller systems and architecture.
4. Communication reduces number of pins necessary to interface with particular modules.

When speaking of communication there are various terms whose definitions are important to have in mind in order to understand the nature of the communication that is occurring:

Parallel/Serial: The key difference between Serial and Parallel Communication is that in serial communication data transmission occurs bit by bit at a time while in parallel communication multiple bits transmit at a time.



Serial Communication



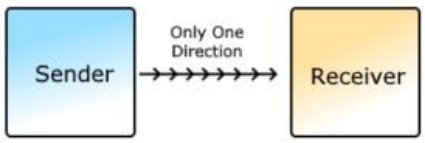
Parallel Communication

Simplex/half duplex/full duplex/:

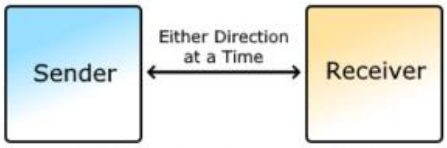
Simplex: In simplex transmission mode, the communication between sender and receiver occurs in only one direction. The sender can only send the data, and the receiver can only receive the data. The receiver cannot reply to the sender.
Example: To take a keyboard / monitor relationship as an example, the keyboard can only send the input to the monitor, and the monitor can only receive the input and display it on the screen. The monitor cannot reply, or send any feedback, to the keyboard.

Half Duplex: The communication between sender and receiver occurs in both directions in half duplex transmission, but only one at a time. The sender and receiver can both send and receive the information, but only one is allowed to send at any given time. Half duplex is still considered a one-way road, in which a vehicle traveling in the opposite direction of the traffic has to wait till the road is empty before it can pass through.
Example: For example, in walkie-talkies, the speakers at both ends can speak, but they have to speak one by one. They cannot speak simultaneously.

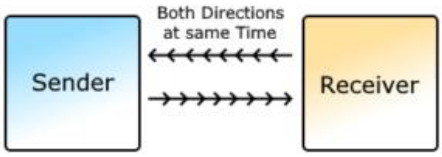
Full Duplex: In full duplex transmission mode, the communication between sender and receiver can occur simultaneously. The sender and receiver can both transmit and receive at the same time. Full duplex transmission mode is like a two-way road, in which traffic can flow in both directions at the same time.
Example: For example, in a telephone conversation, two people communicate, and both are free to speak and listen at the same time.



Simplex Communication



Half Duplex Communication



Full Duplex Communication

Synchronous/ asynchronous:

Synchronous: In this type of serial communication, the sender and receiver are synchronized using an external synchronization clock. Since the data being transmitted has no gaps between them, the receiver identifies the starting of character or data using a common clock pulse. This synchronous clock pulse provides a constant time interval between the data transmission which depends on the frequency of the clock pulse.

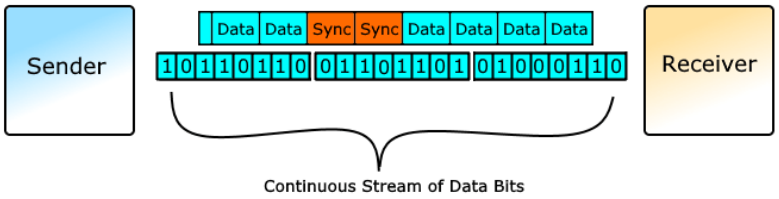
The synchronous transmission needs an extra channel to transmit the clock signal or generate a local internal clock pulse using the information sent by the transmitter. The transmitter also sends **synchronous idle characters** "sync" between the data to resynchronize their clock signals and keep them in sync otherwise the clock pulses may get desynchronized. This phenomenon is called a **master-slave** configuration where the transmitter is master and controls the function of the receiver (slave).The data is continuously transmitted in the form of **blocks** or **frames** (a sequence of bits) and it does not need to transmit any parity bits to identify the start and stop bit of the data. Therefore, the synchronous transmission is fast and reliable for transmitting a large amount of data.

Since the data transmission is continuous, it allows the **full-duplex** mode of communication where the data can flow in both directions at the same time. This offers a real-time communication feature available in live conference, voice and video calling applications.

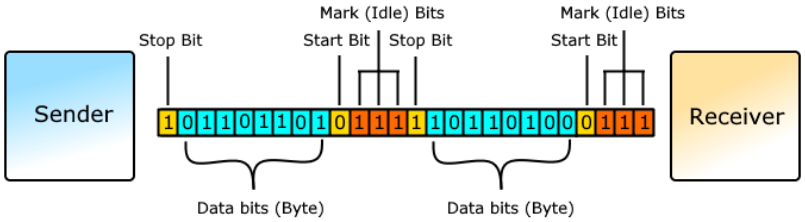
Asynchronous: In this type of serial data transmission, the parity bits at the start and end of the character or bytes are added to identify the beginning and end of the data. Therefore it does not need any external synchronization clock which is why it is called **Asynchronous transmission**. Before sending the data, **mark bits** (idle bits, a sequence of 1's) are sent to the receiver followed by a **start bit 0**, to notify the receiver of incoming data and at the end of a byte, a **stop bit 1** is added to identify the end of the byte. This allows the data to be clear of any errors and easy to read after reassembling at the receiver using the same protocol as the transmitter.

Each byte of data has a **gap** between them due to the parity start and stop bits as well as the mark bits between two consecutive bytes. Due to an increase in the number of bits that need to be transmitted (total of 10 bits instead of 8 bits); the transmission rate of the Asynchronous transmission is relatively **slower** than the Synchronous transmission. It sends **one byte of data** at a time as opposes to the synchronous transmission which can send the whole frame of data.

This form of transmission is **half-duplex**, where the data flows in only one direction at a single time. It is used for transmitting large documents, emails, radio, text messages, etc. where the letters (each letter is a character or a byte) are sent one by one to the receiver.



Synchronous Transmission



Asynchronous Transmission

Protocol: A communication protocol is a system of rules that allow two or more entities of a communications system to transmit information via any kind of variation of a physical quantity. The protocol defines the rules, syntax, semantics and synchronization of communication and possible error recovery methods. Protocols may be implemented by hardware, software, or a combination of both.

Handshaking: In communication, a handshake is an automated process of negotiation between two participants (example, 2 microcontrollers) through the exchange of information that establishes the protocols of a communication link at the start of the communication, before full communication begins. The handshaking process usually takes place in order to establish rules for communication when a computer attempts to communicate with another device. Signals are usually exchanged between two devices to establish a communication link.

Data framing: A frame is a digital data transmission unit in embedded communication. In packet switched systems, a frame is a simple container for a single network packet.

Bus: In communication a bus/data highway is a communication system that transfers data between components inside a computer, or between computers. This expression covers all related hardware components (wire, optical fiber, etc.) and software, including communication protocols.

Choosing a Communication System:

Similar to how we chose whether to drive a nail into wood with a rock or a hammer, the choice of communication method depends on various factors. Both means might be useful depending on the situation. If you are in a well-equipped workshop, with the knowledge and permission to access a hammer, it is the ideal choice. Similarly, if you are in the middle of nowhere without the access of a hammer where there is a time constraint, a rock is ideal in this case.

For example it is common knowledge that CAN BUS is best suited for automobile applications due to its reliability and robustness, hence using I2C or SPI would be at bad choice for this application. Another example would be for networking applications, Ethernet protocol is best suited for this as we know and USB would not be able to cope as network protocol. To transfer massive amount of data such as images or video, you will learn why you cannot use UART but rather PCIe provides you more than enough bandwidth to transfer up to 4K video.

Here are some factors to think about when deciding on which communication method to use.

1. Type of devices used within the communication; capabilities, features etc.
2. Number of devices to be used
3. Amount of data being transported
4. Distance of communication required
5. Speed of communication required
6. Precision of the communication required etc.

Moreover communication protocols are associated with physical layer describing the signals incorporated, signal strength, hand-shaking mechanism, bus arbitration, device addressing, wired or wireless, data lines etc.

There are various forms of communication in embedded systems. They include:

1. **General Purpose Input and Output (GPIO):** The setting GPIO pins whether HIGH or LOW, can be used for commute between 2 devices, a good example is the use of encoder pulses to communicate speed and directionality of a motor.
2. **Universal synchronous/asynchronous receiver-transmitter (USART/UART):** A UART is usually an individual (or part of an) integrated circuit (IC) used for serial communications over a computer or peripheral device serial port. One or more UART peripherals are commonly integrated in microcontroller chips. A related device, the universal synchronous and asynchronous receiver-transmitter (USART) also supports synchronous operation.
3. **Inter-Integrated Circuit / Two Wire Interface (I2C/TWI):** I2C is an extremely common protocol integrated into many products. It allows serial communications between many devices over just two wires. In this tutorial we will cover how I2C works and show some real world examples. If you plan on linking multiple Arduinos or connecting an Arduino to a Raspberry Pi, it's an incredibly useful tool to have in your pocket. You'll also need it for projects that use barometric pressure sensors, or accelerometers, or even OLED displays!
4. **Serial Peripheral Interface (SPI):** SPI devices communicate in full duplex mode using a master-slave architecture with a single master. The master device originates the frame for reading and writing. Multiple slave-devices are supported through selection with individual slave select (SS), sometimes called chip select (CS), lines. Sometimes SPI is called a four-wire serial bus, contrasting with three-, two-, and one-wire serial buses. The SPI may be accurately described as a synchronous serial interface. SPI is one master and multi slave communication.

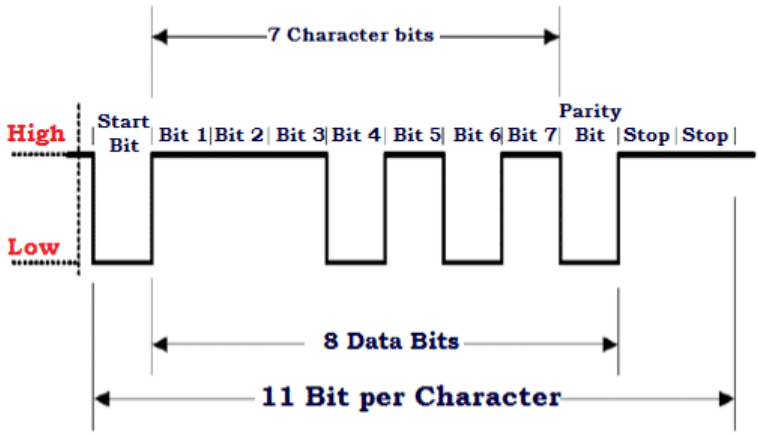
5. USB
6. RJ45/Ethernet
7. Wireless
8. CAN BUS
9. PCIe

UART Communication Protocol

Universal Asynchronous Receiver/Transmitter (UART) is not a communication protocol but just a physical piece of hardware which converts parallel data into serial data. Its main purpose is to transmit and receive data serially. UART is also two-wired i.e., the serial data is handled by Tx (Transmitter) and Rx (Receiver) pins.

UART transmits data asynchronously, which induces that no clock signal is associated in transmitting and receiving data. Instead of clock signal, UART embed start and stop bits with actual data bits, which defines the start and end of data packet.

When receiver end detects the start bit, it starts to read the data bits at specific baud rate meaning both transmitting and receiving peripherals should work under same baud rate. UART works under full duplex communication mode meaning it transmits or receives at a time.

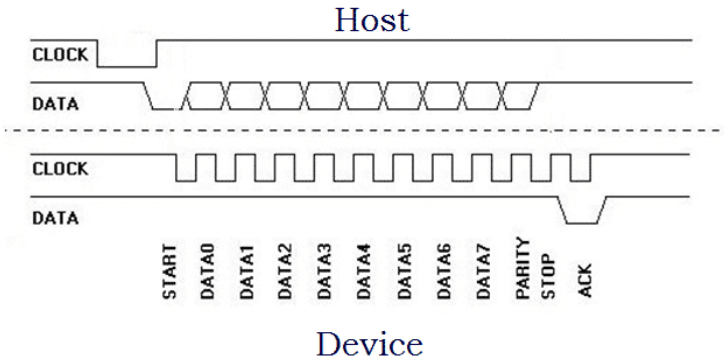


USART Communication Protocol

Universal **S**ynchronous **A**synchronous **R**eceiver/**T**ransmitter (USART) is identical to that of UART with only added functionality synchronous. That is, the transmitter will generate a clock signal which will be recovered at the receiver end from the data stream transmitted without knowing baud rate ahead.

UART works under full duplex communication mode meaning it can transmit and receive data at same time.

USART is half duplex and encompass the abilities of UART, which enables application of both depending on the applications



area.

Advantages of UART/ USART Communication Protocol

- Clock signal is not required
- Cost effective
- Uses parity bit for error detection
- Requires only 2 wires for data communication

Disadvantages of UART/ USART Communication Protocol

- Doesn't support multiple master slave functionality
- Baud rate of communicating UART should be within 10 percent of each other

I2C Communication Protocols

Inter Integrated Circuit (I2C) is a serial communication protocol developed by Philips Semiconductors. The main purpose of this protocol is to provide easiness to connect peripheral chips with microcontroller. In embedded systems, all peripheral devices are connected as memory mapped devices to the microcontroller.

I2C necessitates two wires SDA (Serial Data Line) and SCL (Serial Clock Line) to carry information between devices. These two active wires are said to be bidirectional.

I2C protocol is a master to slave communication protocol. Each slave is been provided with unique address. In order to establish communication, master device initially sends the target slave address along with R/W (Read/Write) flag. The corresponding slave device will move into active mode leaving other devices in off state.

Once the slave device is ready, communication starts between master and slave devices. One bit acknowledgment is replied by the receiver if transmitter transmits 1 byte (8 bits) of data. A stop condition is issued at the end of communication between devices.

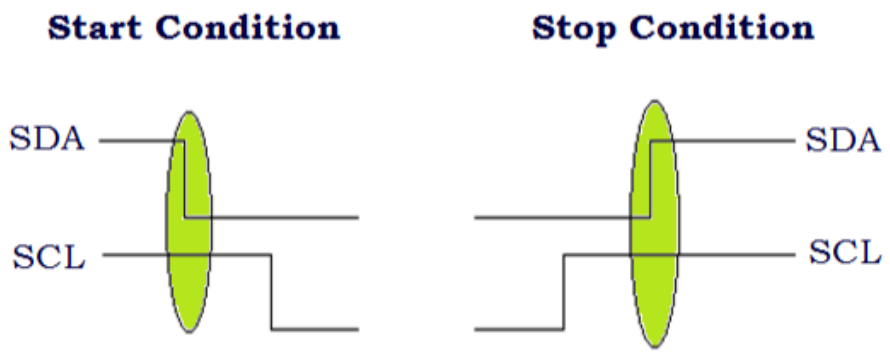


Fig. 8 – Start and Stop Condition of I2C Communication Protocols

Advantages of I2C Communication Protocols

The advantages of I2C Communication Protocols are as follows:

- Provides good communication between onboard devices which are accessed infrequently
- Addressing mechanism eases master slave communication
- Cost and circuit complexity does not end up on number of devices

Disadvantages of I2C Communication Protocols

The biggest disadvantage of I2C Communication Protocols is its limited speed.

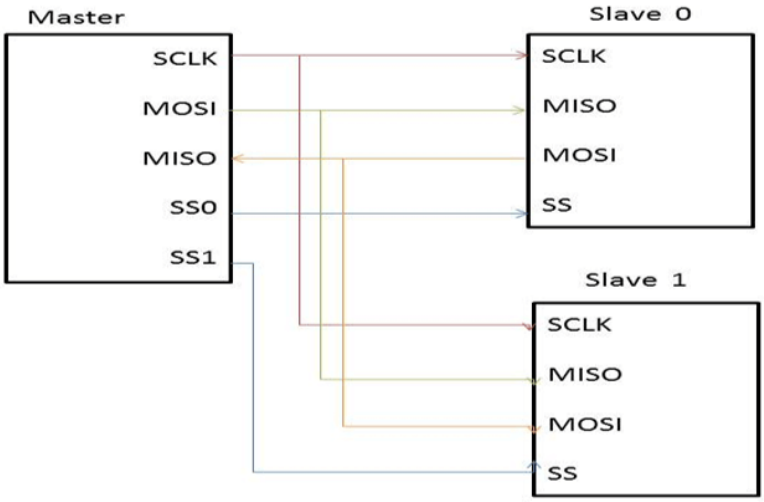
Serial Peripheral Interface (SPI) Communication Protocols

SPI (**S**erial **P**eripheral **I**nterface) is one of the serial communication protocol developed by Motorola. It is a 4-wire protocol namely MOSI (**M**aster **O**ut **S**lave **I**n), MISO (**M**aster **I**n **S**lave **O**ut, SS (**S**lave **S**elect), and SCLK (Serial Clock).

As I2C protocol, SPI is also a master to slave communication protocol. In SPI, the master device first configures the clock at a particular frequency. Furthermore the SS line is used to select the appropriate slave by pulling the SS line low where it is normally held high.

The communication is established between the selected slave and the master device as soon as appropriate slave device is selected.

SPI is a full duplex communication protocol. SPI doesn't limit data transfer to 8 bit words.



SPI Communication Protocols

Advantages of SPI Communication Protocols

- Faster than asynchronous serial communication protocol.
- Support multiple slaves connectivity.
- Universally accepted protocol and low cost.

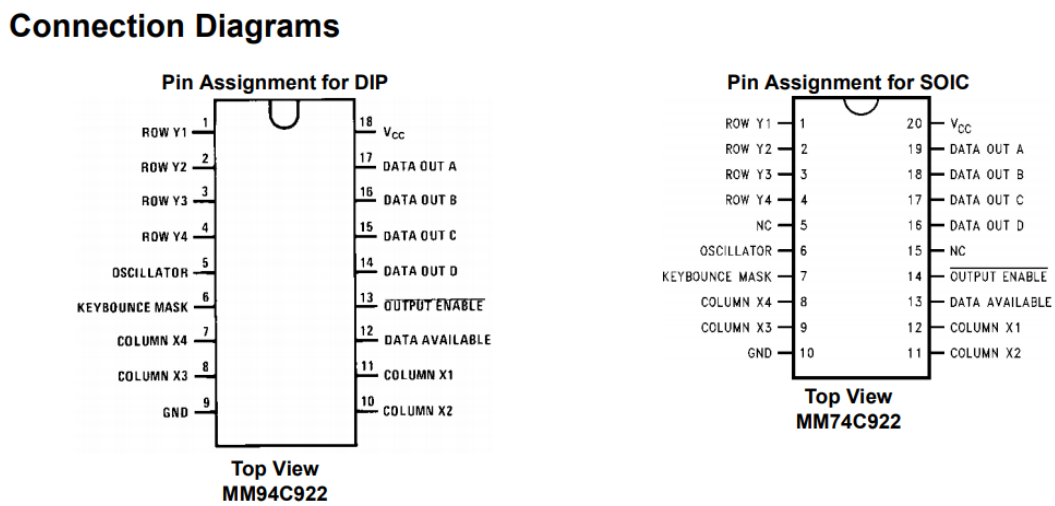
Disadvantages of SPI Communication Protocol

- Requires more wires than other communication protocols.
- Master device should control all slave communications (slave-slave communication is impossible).
- Numerous slave devices leads to circuit complexity.

Keypad interfacing using 16-key Encoder: MM74C922

Plain interfacing of keypads with microcontrollers does take quite some skill level in programming and an increased number of microcontroller pins. To make keypad interfacing more robust and easier, key encoder ICs are normally used. One common one is the MM74C922.

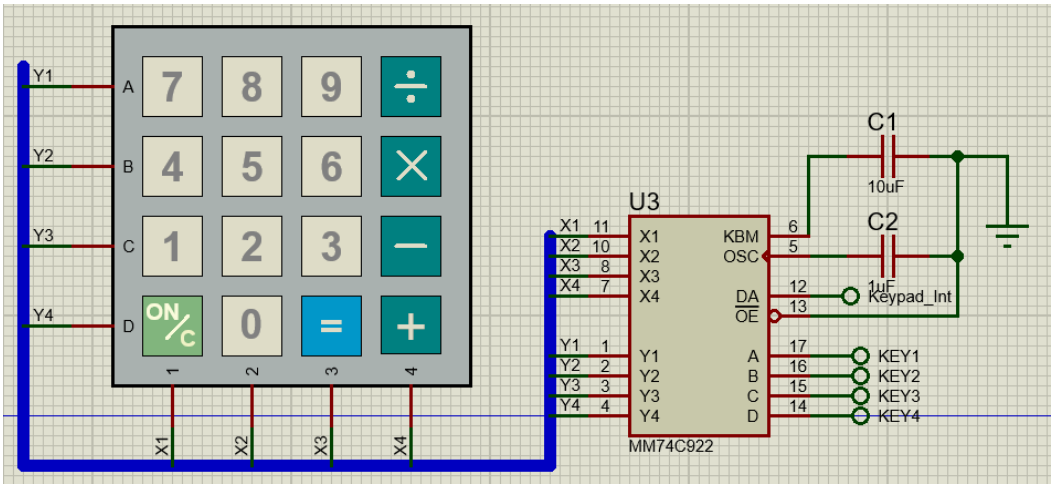
The MM74C922 CMOS key encoders provide all the necessary logic to fully encode an array of SPST switches. The circuit also includes debouncing functionality and has an internal register that keeps the data of the last key pressed, even when the key has been depressed. The chip also has pull-up devices that permit switches with up to 50 kΩ on resistance to be used.



Connection Circuit and Working Logic:

First of all, connect all input keys in the matrix model. It consists of row and column. Which each row and column have four rows, as shown in the circuit below. The pins KEY_1 – KEY_4 are connected to normal GPIO pins. In this case PB0-PB3. The Keypad_int pin is connected to an interrupt enabled pin; in this case PD2.

The IC scans the keypad and every key that is pressed (16 keys), is represented by a unique 4 bit digit transmitted through the ABCD wires (0000 to 1111). Every time a new key is registered, a pulse is sent to the Data Available (DA) pin which can be used to alert the system of a new keypress. This interrupt signal is also called a 'Strobe signal'. When a key has been depressed, the output binary at pin 14 to pin 17 will still hold in all time, until a new key is pressed.



Keypad module code:

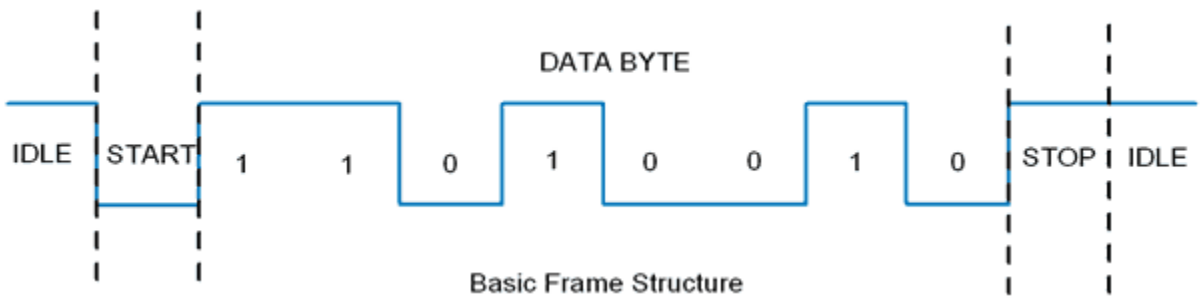
```
/* Keypad array holding the keys in a grid arrangement*/
unsigned char keypad[4][4] = {{'7','8','9','/'},
       {'4','5','6','*'},
       {'1','2','3','-'},
       {'C','0','=','+'}};

/* Function that checks the key that has been pressed on the keypad*/
unsigned char check_Keypad(char input_val){
       int row = input_val/4;
       int col = input_val%4;
       if((input_val>= 0) & (input_val<16)) return (keypad[row][col]);
       else return 0;
}
```

4.1: USART/UART Communication:

AVR ATmega has flexible USART, which can be used for serial communication with other devices like computer, serial GSM, GPS modules etc. Before beginning with AVR USART, we will walk though basics of serial communication.

Serial data framing

While sending/receiving data, some bits are added for the purpose of knowing beginning/ending of data etc. commonly used structure is: 8 data bits, 1 start bit (logic 0) and 1 stop bit (logic 1), as shown:



There are also other supported frame formats available in UART, like parity bit, variable data bits (5-9 data bits).

Speed (Baud rate)

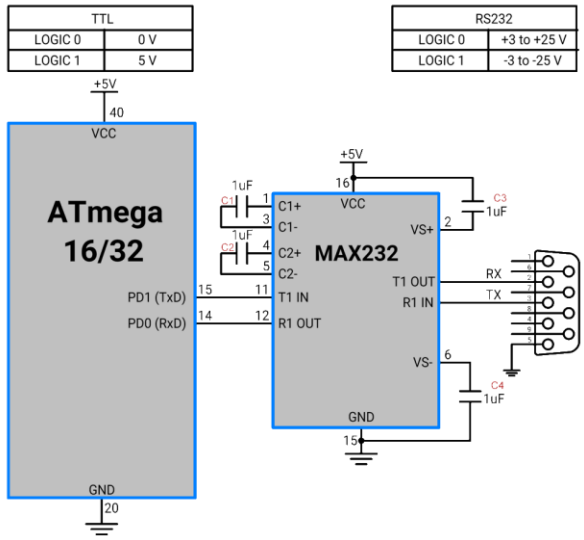
As we know the bit rate is "Number of bits per seconds (bps)", also known as Baud rate in Binary system. Normally this defines how fast the serial line is. There are some standard baud rates defined e.g. 1200, 2400, 4800, 19200, 115200 bps etc. Normally 9600 bps is used where speed is not a critical issue.

Voltage Conversion:

Normally in USART, we only need Tx (Transmit), Rx(Receive) and GND wires.

- AVR ATmega USART has TTL voltage level which are 0 v for logic 0 and 5 v for logic 1.
- In computers and most of the old devices RS232 protocol is used for serial communication, where normally 9 pin 'D' shape connector is used. RS232 serial communication has different voltage levels than ATmega serial communication i.e. +3 v to +25 v for logic zero and -3 v to -25 v for logic 1.

- So to communicate with RS232 protocol, we need to use voltage level converter like MAX232 IC.



AVR basic Registers

1. **UDR: USART Data Register**: It has basically two registers, one is Tx. Byte and other is Rx Byte. Both shares the same UDR register. Do remember that, when we write to the UDR reg. Tx buffer will get written and when we read from this register, Rx Buffer will get read. Buffer uses FIFO shift register to transmit the data.
2. **UCSRA**: USART Control and Status Register A. As the name suggests, is used for control and status flags. In a similar fashion, there are two more USART control and status registers, namely UCSRB and UCSRC.
3. **UBRR**: USART Baud Rate Register, this is 16-bit register used for setting baud rate.

UCSRA: USART Control and Status Register A

7	6	5	4	3	2	1	0
RXC	TXC	UDRE	FE	DOR	PE	U2X	MPCM

- **Bit 7 – RXC:** USART Receive Complete

This flag bit is set when there is unread data in UDR. The RXC Flag can be used to generate a Receive Complete interrupt.

- **Bit 6 – TXC:** USART Transmit Complete

This flag bit is set when the entire frame from Tx Buffer is shifted out and there is no new data currently present in the transmit buffer (UDR). The TXC Flag bit is automatically cleared when a transmit complete interrupt is executed, or it can be cleared by writing a one to its bit location. The TXC Flag can generate a Transmit Complete interrupt.

- **Bit 5 – UDRE:** USART Data Register Empty

If UDRE is one, the buffer is empty which indicates the transmit buffer (UDR) is ready to receive new data. The UDRE Flag can generate a Data Register empty Interrupt. UDRE is set after a reset to indicate that the transmitter is ready.

- **Bit 4 – FE:** Frame Error

- **Bit 3 – DOR:** Data OverRun

This bit is set if a Data OverRun condition is detected. A Data OverRun occurs when the receive buffer is full (two characters) and a new character is waiting in the receive Shift Register.

- **Bit 2 – PE:** Parity Error
- **Bit 1 – U2X:** Double the USART Transmission Speed
- **Bit 0 – MPCM:** Multi-processor Communication Mode

UCSRB: USART Control and Status Register B

7	6	5	4	3	2	1	0
RXCIE	TXCIE	UDRIE	RXEN	TXEN	UCSZ2	RXB8	TXB8

- **Bit 7 – RXCIE:** RX Complete Interrupt Enable
  Writing one to this bit enables interrupt on the RXC Flag.
- **Bit 6 – TXCIE:** TX Complete Interrupt Enable
  Writing one to this bit enables interrupt on the TXC Flag.
- **Bit 5 – UDRIE:** USART Data Register Empty Interrupt Enable
  Writing one to this bit enables interrupt on the UDRE Flag.
- **Bit 4 – RXEN:** Receiver Enable
  Writing one to this bit enables the USART Receiver.
- **Bit 3 – TXEN:** Transmitter Enable
  Writing one to this bit enables the USART Transmitter.
- **Bit 2 – UCSZ2:** Character Size

The UCSZ2 bits combined with the UCSZ1:0 bit in UCSRC sets the number of data bits (Character Size) in a frame the receiver and transmitter use.

- **Bit 1 – RXB8:** Receive Data Bit 8
- **Bit 0 – TXB8:** Transmit Data Bit 8

UCSRC: USART Control and Status Register C

7	6	5	4	3	2	1	0
URSEL	UMSEL	UPM1	UPM0	USBS	USCZ1	USCZ0	UCPOL

- **Bit 7 – URSEL:** Register Select

This bit selects between accessing the **UCSRC** or the **UBRRH** Register, as both register shares same address. The URSEL must be one when writing the UCSRC or else data will be written in UBRRH register.

- **Bit 6 – UMSEL:** USART Mode Select
  This bit selects between Asynchronous and Synchronous mode of operation.
  0 = Asynchronous Operation
  1 = Synchronous Operation
- **Bit 5:4 – UPM1:0**: Parity Mode

These bits enable and set type of parity generation and check. If parity a mismatch is detected, the PE Flag in UCSRA will be set.

UPM1	**UPM0**	**Parity Mode**
0	0	Disabled
0	1	Reserved

UPM1	UPM0	Parity Mode
1	0	Enabled, Even Parity
1	1	Enabled, Odd Parity

- **Bit 3 – USBS**: Stop Bit Select

This bit selects the number of Stop Bits to be inserted by the Transmitter. The Receiver ignores this setting.

0 = 1-bit
1 = 2-bit

- **Bit 2:1 – UCSZ1:0**: Character Size

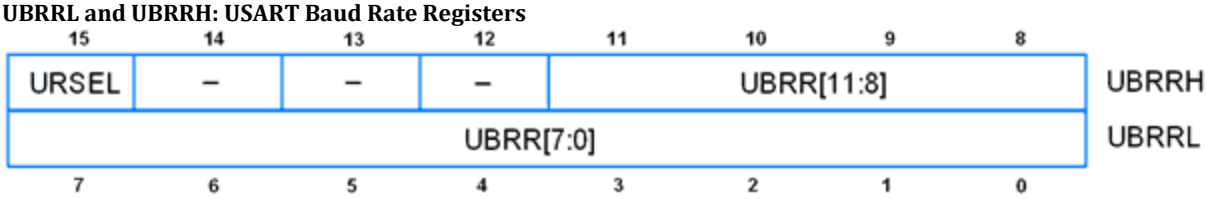
The UCSZ1:0 bits combined with the UCSZ2 bit in UCSRB sets the number of data bits (Character Size) in a frame the Receiver and Transmitter use.

UCSZ2	UCSZ1	UCSZ0	Character Size
0	0	0	5-bit
0	0	1	6-bit
0	1	0	7-bit
0	**1**	**1**	**8-bit**
1	0	0	Reserved
1	0	1	Reserved
1	1	0	Reserved
1	1	1	9-bit

- **Bit 0 – UCPOL:** Clock Polarity

This bit is used for Synchronous mode only. Write this bit to zero when Asynchronous mode is used.

UBRRL and UBRRH: USART Baud Rate Registers



- **Bit 15 – URSEL:** Register Select

This bit selects between accessing the **UCSRC** or the **UBRRH** Register, as both register shares same address. The URSEL must be one when writing the UCSRC or else data will be written in UBRRH register.

- **Bit 11:0 – UBRR11:0:** USART Baud Rate Register.

Used to define baud rate

$$\mathrm{UBBR} = \frac{\mathrm{Fosc}}{16 * \mathrm{BaudRate}} - 1 \qquad \mathrm{BaudRate} = \frac{\mathrm{Fosc}}{16 * (\mathrm{UBBR} + 1)}$$

Example: suppose Fosc=8 MHz and required baud rate= 9600 bps.
Then value of **UBRR= 51.088 i.e. 51.**

We can also set this value by c code using pre-processor macro as follow.

```
#define F_CPU 8000000UL                /* Define frequency here its 8MHz */
#define USART_BAUDRATE 9600
#define BAUD_PRESCALE (((F_CPU / (USART_BAUDRATE * 16UL))) - 1)
```

BAUD_PRESCALE is the value that we have to load in UBRR register to set defined baud rate.

USART Functions:

For programming the USART, we first need to consult the Atmega32 datasheet. Based on the data sheet information, we need to generate 3 main functions; USART, initialization, receiving a character and sending a character. These form the foundation for other functions for sending and receiving entire strings.

```
/*
 * USART_Functions.c
```

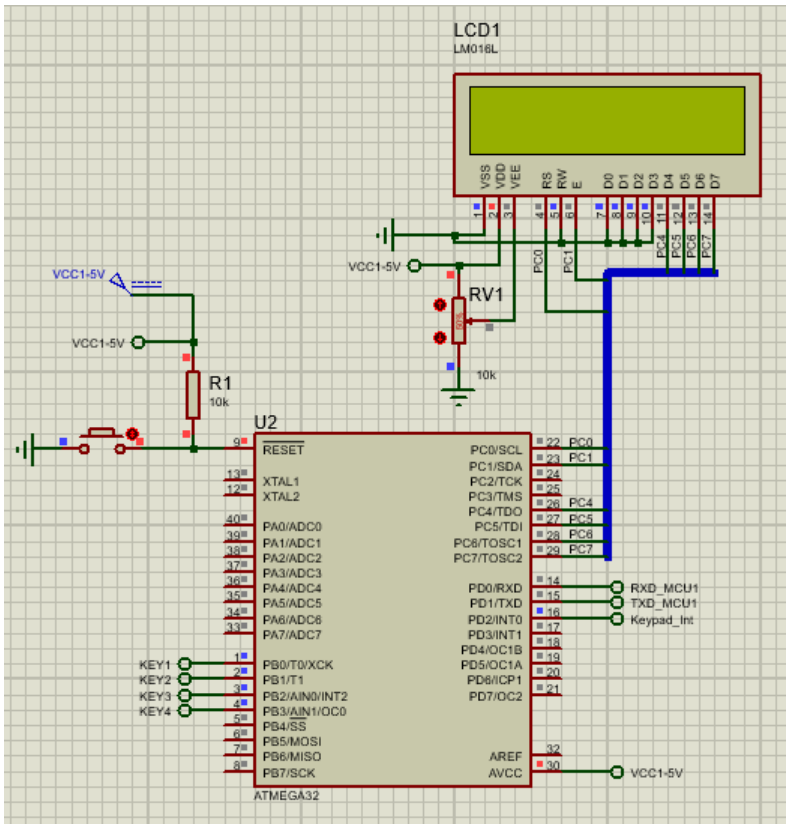
```
 *  Author: Michael Mureithi
 */

#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <stdlib.h>
#include <string.h>

#define BAUDRATE 9600
#define BAUD_PRESCALLER (((F_CPU / (BAUDRATE * 16UL))) - 1)
#define msg_len 8
const char schar[] = "[" ;
const char cchar[] = "]" ;


/* Initialize USART */
void USART_Init(void){
        UBRRL = (uint8_t)(BAUD_PRESCALLER);
        UBRRH = (uint8_t)(BAUD_PRESCALLER>>8);
        UCSRB = (1<<RXEN)|(1<<TXEN)|(1<<RXCIE);
        UCSRC = (1<<UCSZ0)|(1<<UCSZ1)|(1<<URSEL);
}

/* Function to receive byte/char */
char USART_receive(void){
        //Wait until the RXC flag is set to high then return the contents of UDR
        while(!(UCSRA & (1<<RXC)));
        return UDR;
}

/* Function to send byte/char */
void USART_send(char data){
        while(!(UCSRA & (1<<UDRE)));
        UDR = data;
}

/* Function to receive string */
int USART_readString(char *buff){
        int msgSize = 0;
        for(msgSize = 0; msgSize < msg_len; msgSize++){
                char hold = USART_receive();
                if(hold == cchar[0]) break;
                buff[msgSize] = hold;
                _delay_ms(1);
        }
        return msgSize;
}

/* Function to Send string */
void USART_putstring(char *StringPtr){
        int count = 0;
        char package[msg_len+2];
        strcpy(package, schar);
        strcat(package, StringPtr);
        strcat(package, cchar);
        do{
                USART_send(package[count]);
                count++;
        }while((count<msg_len+2) & (package[(count-1)] != cchar[0]));
}
```

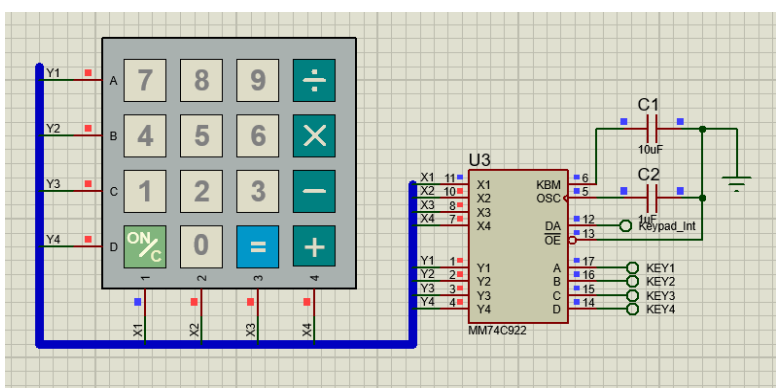
Exercise Task:

To fully understand the usefulness of USART, we will implement a 2 way communication channel between 2 ATmega32 microcontrollers (MCU1 and MCU2). MCU1 will act as the master IC and will feature a keypad interfaced using a MM74C922. A user will be able to send a command message to MCU 2 by typing it on the keypad and pressing the ON/C button to send. MCU1 will also receive sensor readings from MCU 2 and display them on an LCD screen.

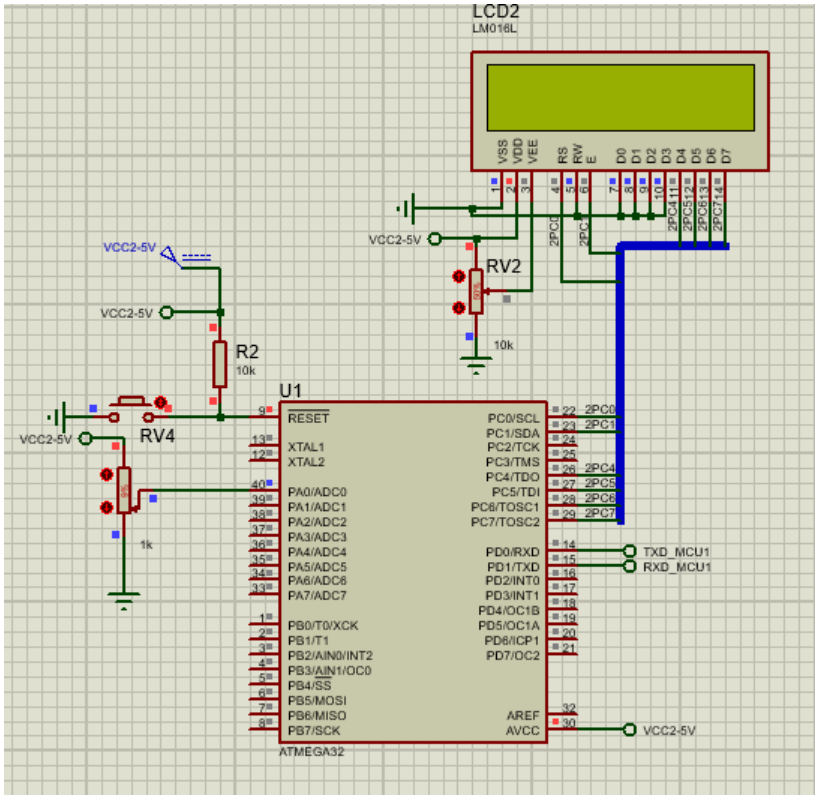
MCU2 will feature a sensor (potentiometer) and an LCD. It will send the value of the potentiometer to MCU1 and will display the value of any MCU 1 command sent via USART on the LCD display.

Circuit Implementation:





MCU 1 with LCD and Keypad connection using the MMC74C922



MCU 2 with sensor (Potentiometer) and LCD

Code Section:

.

```
/*
 * USART_MCU1.c
*/

#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <stdlib.h>
#include <avr/interrupt.h>
#include "USART_Header.h"

/* Variables for data framing */
#define msg_len 8
#define snd_len 8
const char start[] = "[" ;

int msgNo = 0;
int msgSize = 0;
char checking = 1;
char sample = 0;

char read_msg[msg_len];
char send_msg[snd_len];

int keycount = -1;
```

```c
/* Utility functions to clear the values in buffer variables */
void clear_read(char *buff){
	for (int i = 0; i < msg_len; i++){
		buff[i]= 0;
	}
}

void clear_send(char *buff){
	for (int i = 0; i < snd_len; i++){
		buff[i]= 0;
	}
}

/*ISR function; run whenever there is a new message received on the RX register*/
ISR(USART_RXC_vect){
	// Running the USART_receive() function removes the information from the register
	// We use the checking variable to know when to sample data or read directly.
	if(checking){
		sample = USART_receive();
	}
	// If the sampled data is the beginning of a new message, read until the
	// end of the message is identified.
	if (sample == start[0]){
		clear_read(read_msg);
		checking = 0;
		msgSize = USART_readString(read_msg);
		checking = 1;
		msgNo++;
	}
}

/*ISR function: run whenever there is a new key press from the MMC74C922*/
ISR(INT0_vect){
	// Read the value from the keypad connected pins
	char value = PINB & (0x0F);
	unsigned char keycheck = check_Keypad(value);

	// If the key pressed is C, send a message to MCU2
	if (keycheck == 'C'){
		USART_putstring(send_msg);
		keycount = 0;
		clear_send(send_msg);
	}
	else{
		if(keycount < 0){
			keycount++;
		}else if((keycount>= 0) && (keycount < 8)){
			send_msg[keycount] = keycheck;
			keycount++;
			if(keycount == 8) keycount = 0;
		}
	}
}

/* Setup function for the Keypad */
void keypad_Init(){
	DDRD |= (0<<PD2);	/* PORTD as input */
	PORTD |= (1<<PD2);	/* Activate pull up resistor high */

	/* Interrupt setup */
	GICR = 1<<INT0;		/* Enable INT0*/
```

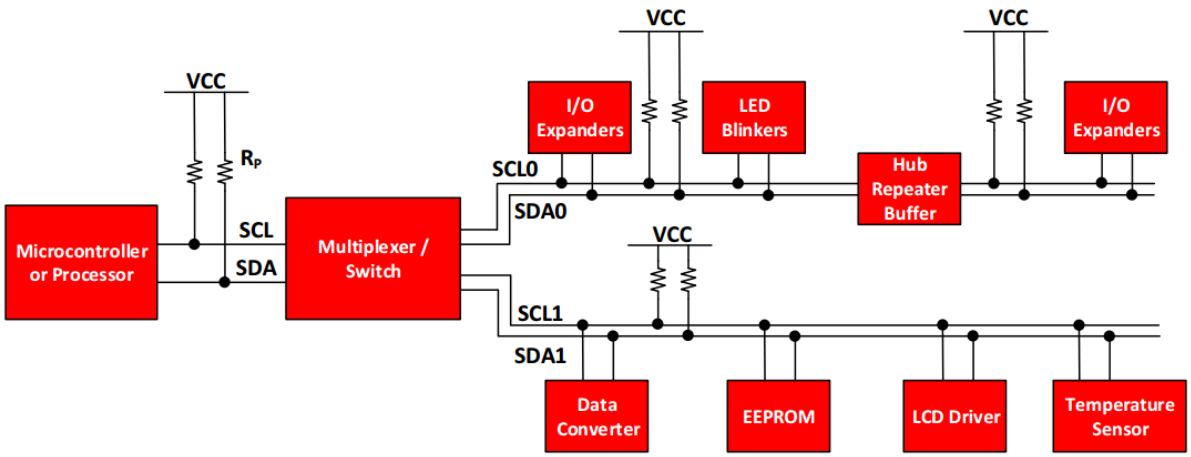
```c
	MCUCR = 1<<ISC01 | 1<<ISC00;  /* Trigger INT0 on rising edge */
	sei();                        /* Enable Global Interrupt */
}

/* Function to display the relevant message on the LCD */
void display_Content(){
	char buffer[8];
	LCD_Cmd(0x80);
	LCD_String("MM:");
	LCD_String(send_msg);
	LCD_String_xy(0,11,":");
	LCD_Cmd(0xC0);
	LCD_String("SM(");
	itoa(msgNo, buffer, 10);
	LCD_String(buffer);
	LCD_String("): ");
	LCD_String(read_msg);
}


int main(void){
	sei();
	USART_Init();
	keypad_Init();
	LCD_Init();
	while(1){
		display_Content();
	}
	return 0;
}
```

```c
/*
 * USART_MCU2.c
 * Author : Michael Mureithi
 */

#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <stdlib.h>
#include <string.h>
#include <avr/interrupt.h>
#include "USART_Header.h"

/* Variables for data framing */
#define msg_len 8
char buffer[8];
char buffer1[8];
char buffer2[8];

const char start[] = "[" ;
char read_msg[msg_len];

int msgNo = 0;
int msgSize = 0;
char checking = 1;
char sample = 0;
int send_count = 0;

/* Utility functions to clear the values in buffer variables */
void clear_read(char *buff){
	for (int i = 0; i < msg_len; i++){
```

```
                buff[i]=0;
        }
}

void Timer_Init(){
        /* Interrupt setup */
        TIMSK=(1<<TOIE0);        /* Enable Timer0 overflow interrupts */
        TCNT0 = 0x00;            /* Load TCNT0, count for 32ms*/
        TCCR0 = (1<<CS02) | (1<<CS00);   /* Start timer0 with 1024 pre-scaler*/
}

/*ISR function; run whenever there is a new message received on the RX register*/
ISR(USART_RXC_vect){
        // Running the USART_receive() function removes the information from the register
        // We use the checking variable to know when to sample data or read directly.
        if(checking){
                sample = USART_receive();
        }
        // If the sampled data is the beginning of a new message, read until the
        // end of the message is identified.
        if (sample == start[0]){
                clear_read(read_msg);
                checking = 0;
                msgSize = USART_readString(read_msg);
                checking = 1;
                msgNo++;
        }
}


/*Interrupt Service Routine for INT0*/
ISR(TIMER0_OVF_vect){
        // The if >= 32 helps send a message every 1 sec as max 8 bit timer time is
        // 32ms thus to achieve 1 sec intervals, we need 32*31 ~= 992ms
        if(send_count >= 31){
                // prepare message package and send.
                char string_code[] = "Pot>";
                int Sensor_Value = ADC_Read(0);
                itoa(Sensor_Value, buffer, 10);
                strcat(string_code, buffer);
                USART_putstring(string_code);
                send_count = 0;
        }
        send_count++;
        TCNT0 = 0x00;            /* Load TCNT0*/
}


int main(void){
        // Call initialization codes
        sei();
        USART_Init();
        ADC_Init();
        LCD_Init();
        Timer_Init();
        while(1){
                // Display content on LCD
                LCD_Cmd(0xC0);
                LCD_String("MM:");
                LCD_String(read_msg);
        }

        return 0;
}
```

4.2: I2C Communication:

The Inter-Integrated Circuit (I^2C) Protocol is a protocol intended to allow multiple "slave" digital integrated circuits ("chips") to communicate with one or more "master" chips. Like the Serial Peripheral Interface (SPI), it is only intended for short distance communications within a single device. Like Asynchronous Serial Interfaces (such as RS-232 or UARTs), it only requires two signal wires to exchange information. **(Reference: Texas Instruments application report: SLVA704 – Understanding the I2C bus)**

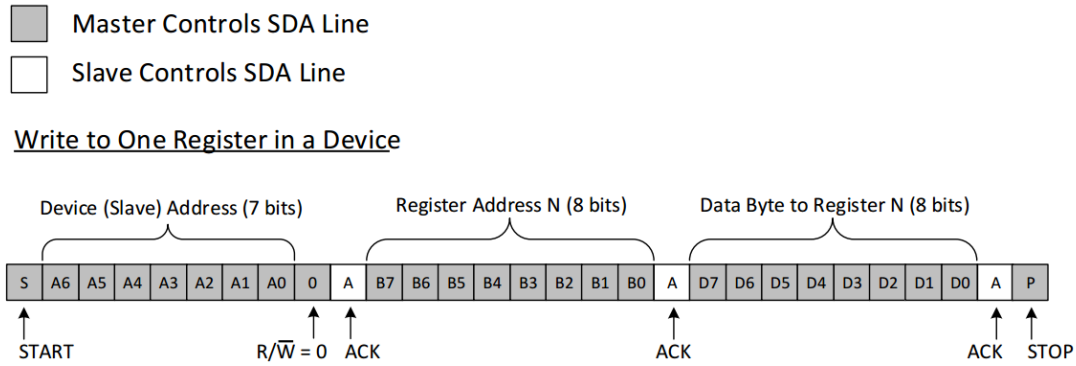


Example I^2C Bus

Communication Flow:

To effectively understand how to write the code, there is need to know what exactly happens during the communication process. In addition, the process is different for reading and writing, thus we will consider what happens when a master wants to sends data to a slave and when a master wants to receive data from a slave. For both cases we will look at what registers and bits the master and slave work with in each case. For communication to happen in both situations i.e. reading and writing data, the master is the device to initiate the communication.

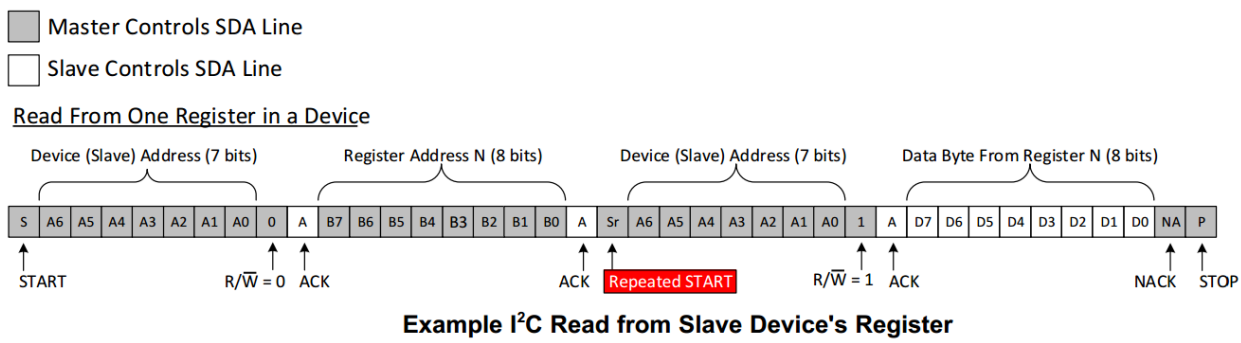
The general procedure for a master to access a slave device is the following:



Example I^2C Write to Slave Device's Register

1. Suppose a master wants to send data to a slave:
   - Master-transmitter sends a **START** condition and addresses the slave-receiver
   - Master-transmitter sends a **R/W bit.** If we are writing this is **0**
   - Slave – receiver sends an **ACK** (acknowledge) bit.

- Master-transmitter sends the slave register to be accessed.
- Slave – receiver sends an **ACK** (acknowledge) bit.
- Master-transmitter sends data to slave-receiver
- Slave – receiver sends an **ACK** (acknowledge) bit.
- Master-transmitter terminates the transfer with a **STOP** condition



Example I²C Read from Slave Device's Register

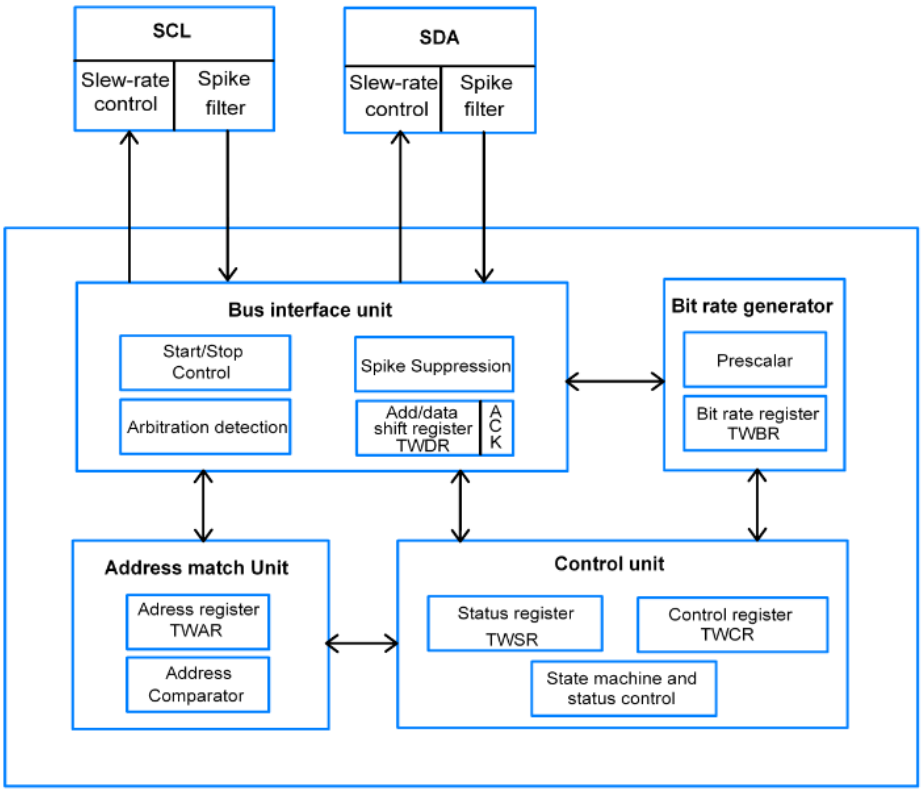
2. If a master wants to receive/read data from a slave:
   - Master-receiver sends a **START** condition and addresses the slave-transmitter
   - Master-receiver sends a **R/W bit.** If we are reading this is **0**
   - Slave – transmitter sends an **ACK** (acknowledge) bit.
   - Master-receiver sends the requested register to read to slave-transmitter
   - Slave – transmitter sends an **ACK** (acknowledge) bit.
   - Master-receiver sends a **REPEATE START** and addresses the slave-transmitter
   - Master-receiver sends a **R/W bit.** If we are reading this is **1**
   - Slave – transmitter sends an **ACK** (acknowledge) bit.
   - Master-receiver sends a **NACK** (not acknowledge) bit.
   - Master-receiver terminates the transfer with a STOP condition

Important Elements:

From the above breakdown, we can observe there is a part to be done by the **Master** device and the **Slave** device. For the master device we require a **START** and **STOP** condition. Within these two condition, you can have a **REPEAT START** condition. We also need to create an **ACK** (acknowledge) and a **NACK** (not acknowledge) functionality for both the Master and Slave devices. We also need to set the **R/W bit,** when we begin the start or the repeat start functions.

The key coding elements will include:

1. **Initialization**:
   This section requires the necessary settings for the communication to be done i.e. communication speed and method of message arrival checking i.e. polling or interrupt based.
2. **Communication Commands:**
   Master Functions: **START, STOP, REPEAT START, NACK, R/W, SEND BYTE, READ BYTE**
   Slave Functions: **ACK, SEND BYTE, READ BYTE**



ATmega32 I2C Module

SDA & SCL pins

- These pins are used to interface the TWI based external peripherals and microcontroller.
- The output drivers contain a slew-rate limiter. The input stages contain a spike suppression unit which removes spikes shorter than 50 ns.

Bus interface unit

- Bus interface unit contains Start/Stop control which is responsible for generation and detection of START, REPEATED START and STOP conditions.
- TWDR add/data shift register contain data to be transmitted and received.
- ACK bit receive ack/nack in transmitter mode and it is generated through software in receiving mode.
- Spike suppression unit take care of spikes whereas arbitration detection continuous monitor bus status and inform control unit about it.

Address match unit

In slave mode, Address match unit receive incoming 7-bit address and compare with address in TWAR (Two Wire Address Register) register to check whether it matches or not and upon match occur it intimate to control unit to take necessary action. It also considers general call address if TWGCE bit in TWAR is enabled.

Bit rate generator unit

Bit rate generator unit control the SCL period in master mode to generate SCL frequency. It is calculated by,

SCL frequency = (CPU CLK frequency)/(16+2(TWBR)*4^TWPS)

Where TWPS is a value of a pre-scaler bit in TWSR.

Control unit

- Control unit contain TWSR (TWI status register), TWCR (TWI control register).
- It controls overall process of attention for necessary events, identifying eventswhen occur, TWINT interrupt assertion and update TWSR.
- As long as TWINT flag set SCL held low. TWINT set whenever TWI complete current task.

Let see registers in ATmega32 I2C module

TWBR: TWI Bit Rate Register

TWI bit rate register used in generating SCL frequency while operating in master mode

7	6	5	4	3	2	1	0	
TWBR7	TWBR6	TWBR5	TWBR4	TWBR3	TWBR2	TWBR1	TWBR0	TWBR Register

TWCR:TWI Control Register

TWI control resistor used to control events of all I2C communication.

7	6	5	4	3	2	1	0	
TWINT	TWEA	TWSTA	TWSTO	TWWC	TWEN	–	TWIE	TWCR Register

Bit 7 – TWINT:TWI interrupt

- This bit gets set whenever TWI completes its current event (like start, stop, transmit, receive, etc).
- While I-bit in SREG and TWIE bit in TWCR is enabled then TWI interrupt vector called whenever TWI interrupt occur.
- **TWI interrupt flag must be cleared by software by writing logical one to it.**This bit is not automatically cleared by hardware.

Bit 6 – TWEA:TWI enable acknowledgement bit

- This is TWI acknowledgement enable bit, it is set in receiver mode to generate acknowledgement and cleared in transmit mode.

Bit 5 – TWSTA: TWI START condition bit

- Master device set this bit to generate START condition by monitoring free bus status to take control over TWI bus.

Bit 4 – TWSTO: TWI STOP condition bit

- Master device set this bit to generate STOP condition to leave control over TWI bus.

Bit 3 – TWWC: TWI write collision

- This bit gets set when writing to TWDR register before current transmission not complete i.e. TWINT is low.

Bit 2 – TWEN: TWI enable bit

- This bit set to enables the TWI interface in device and takes control over the I/O pins.

Bit 1 - Reserved

Bit 0 – TWIE: TWI interrupt enable

- This bit is used to enable TWI interrupt routine while I-bit of SREG is set as long as TWINT flag is high.

TWSR: TWI Status Register

7	6	5	4	3	2	1	0	
TWS7	TWS6	TWS5	TWS4	TWS3	——	TWPS1	TWPS0	TWSR Register

Bit 7: Bit 3 - TWS7: TWS3:TWI status bits

- TWI status bits shows the status of TWI control and bus

Bit 1:0 - TWPS1:TWPS0: TWI pre-scaler bits

- TWI pre-scaler bits used in bit rate formula to calculate SCL frequency

TWPS1	TWPS0	Exponent	Pre-scaler value
0	0	0	1
0	1	1	4
1	0	2	16
1	1	3	64

TWDR: TWI Data Register

- TWDR contain data to be transmitted or received.
- It's not writable while TWI is in process of shifting a byte.
- The data remains stable as long as TWINT is set.

7	6	5	4	3	2	1	0	
								TWDR Register

TWAR: TWI Address Register

- TWAR register contain address of TWI unit in slave mode.
- It is mostly used in multi-master system.

7	6	5	4	3	2	1	0	
TWA6	TWA5	TWA4	TWA3	TWA2	TWA1	TWA0	TWGCE	TWAR Register

Bit 7:1 - TWA6: TWA0: TWI address bits

- TWI address bits contain TWI 7-bit address with which it can called by other masters in slave mode.

Bit 0 – TWGCE: TWI general call enable bit

- TWI general call enable bit when set it enable recognition of general call over TWI bus

There are four transmission modes in I2C in which I2C device works.

- When device is Master it works in MT and MR transmission modes.
- And when device is Slave it works in ST and SR transmission modes.

SR No.	Transmission mode	Operation
1	Master Transmitter (MT)	Master device write data to SDA.
2	Master Receiver (MR)	Master device read data from SDA.
3	Slave Transmitter (ST)	Slave device write data to SDA.
4	Slave Receiver (SR)	Slave device read data from SDA.

Let see how to program I2C in Master and Slave mode.

Master Functions:

```
#include "I2C_Master_H_file.h"                                  /* Include I2C header file */

void I2C_Init()                                                 /* I2C initialize function */
{
        TWBR = BITRATE(TWSR = 0x00);                    /* Get bit rate register value by formula */
}


uint8_t I2C_Start(char write_address) /* I2C start function */
{
        uint8_t status;
        /* Declare variable */
        TWCR = (1<<TWSTA)|(1<<TWEN)|(1<<TWINT);/* Enable TWI, generate start condition and clear interrupt flag */
        while (!(TWCR & (1<<TWINT)));           /* Wait until TWI finish its current job (start condition) */

        status = TWSR & 0xF8;                   /* Read TWI status register with masking lower three bits */
        if (status != 0x08)                     /* Check whether start condition transmitted successfully or not? */
        return 0;                               /* If not then return 0 to indicate start condition fail */

        TWDR = write_address;                   /* If yes then write SLA+W in TWI data register */
        TWCR = (1<<TWEN)|(1<<TWINT);            /* Enable TWI and clear interrupt flag */
        while (!(TWCR & (1<<TWINT)));           /* Wait until TWI finish its current job (Write operation) */
        status = TWSR & 0xF8;                   /* Read TWI status register with masking lower three bits */

        if (status == 0x18)                     /* Check whether SLA+W transmitted & ack received or not? */
        return 1;                               /* If yes then return 1 to indicate ack received; ready to accept data*/
        if (status == 0x20)                     /* Check whether SLA+W transmitted & nack received or not? */
        return 2;                               /* If yes then return 2 to indicate nack received i.e. device is busy */
        else
        return 3;                               /* Else return 3 to indicate SLA+W failed */
}

uint8_t I2C_Repeated_Start(char read_address)  /* I2C repeated start function */
{
        uint8_t status;                         /* Declare variable */
        TWCR = (1<<TWSTA)|(1<<TWEN)|(1<<TWINT);/* Enable TWI, generate start condition and clear interrupt flag */
        while (!(TWCR & (1<<TWINT)));           /* Wait until TWI finish its current job (start condition) */
        status = TWSR & 0xF8;                   /* Read TWI status register with masking lower three bits */
        if (status != 0x10)                     /* Check whether repeated start condition transmitted successfully*/
        return 0;                               /* If no then return 0 to indicate repeated start condition fail */
        TWDR = read_address;                    /* If yes then write SLA+R in TWI data register */
        TWCR = (1<<TWEN)|(1<<TWINT);            /* Enable TWI and clear interrupt flag */
        while (!(TWCR & (1<<TWINT)));           /* Wait until TWI finish its current job (Write operation) */
        status = TWSR & 0xF8;                   /* Read TWI status register with masking lower three bits */
        if (status == 0x40)                     /* Check weather SLA+R transmitted & ack received or not? */
        return 1;                               /* If yes then return 1 to indicate ack received */
        if (status == 0x20)                     /* Check weather SLA+R transmitted & nack received or not? */
        return 2;                               /* If yes then return 2 to indicate nack received i.e. device is busy */
        else
        return 3;                               /* Else return 3 to indicate SLA+W failed */
}
```

```c
void I2C_Stop()    /* I2C stop function */
{
        TWCR=(1<<TWSTO)|(1<<TWINT)|(1<<TWEN); /* Enable TWI, generate stop condition and clear interrupt flag */
        while(TWCR & (1<<TWSTO));             /* Wait until stop condition execution */
}

void I2C_Start_Wait(char write_address)       /* I2C start wait function */
{
        uint8_t status;                       /* Declare variable */
        while (1)
        {
                TWCR = (1<<TWSTA)|(1<<TWEN)|(1<<TWINT);/* Enable TWI, generate start condition and clear interrupt
flag */
                while (!(TWCR & (1<<TWINT)));/* Wait until TWI finish its current job (start condition) */
                status = TWSR & 0xF8;         /* Read TWI status register with masking lower three bits */
                if (status != 0x08)           /* Check whether start condition transmitted successfully or not? */
                continue;                     /* If no then continue with start loop again */

                TWDR = write_address;         /* If yes then write SLA+W in TWI data register */
                TWCR = (1<<TWEN)|(1<<TWINT);/* Enable TWI and clear interrupt flag */
                while (!(TWCR & (1<<TWINT)));/* Wait until TWI finish its current job (Write operation) */
                status = TWSR & 0xF8;         /* Read TWI status register with masking lower three bits */
                if (status != 0x18 )          /* Check whether SLA+W transmitted & ack received or not? */
                {
                        I2C_Stop();           /* If not then generate stop condition */
                        continue;             /* continue with start loop again */
                }
                break;                        /* If yes then break loop */
        }
}

uint8_t I2C_Write(char data)                  /* I2C write function */
{
        uint8_t status;                       /* Declare variable */
        TWDR = data;                          /* Copy data in TWI data register */
        TWCR = (1<<TWEN)|(1<<TWINT);          /* Enable TWI and clear interrupt flag */
        while (!(TWCR & (1<<TWINT)));         /* Wait until TWI finish its current job (Write operation) */
        status = TWSR & 0xF8;                 /* Read TWI status register with masking lower three bits */
        if (status == 0x28)                   /* Check weather data transmitted & ack received or not? */
        return 0;                             /* If yes then return 0 to indicate ack received */
        if (status == 0x30)                   /* Check weather data transmitted & nack received or not? */
        return 1;                             /* If yes then return 1 to indicate nack received */
        else
        return 2;                             /* Else return 2 to indicate data transmission failed */
}

char I2C_Read_Ack()                           /* I2C read ack function */
{
        TWCR=(1<<TWEN)|(1<<TWINT)|(1<<TWEA);  /* Enable TWI, generation of ack and clear interrupt flag */
        while (!(TWCR & (1<<TWINT)));         /* Wait until TWI finish its current job (read operation) */
        return TWDR;                          /* Return received data */
}

char I2C_Read_Nack()                          /* I2C read nack function */
{
        TWCR=(1<<TWEN)|(1<<TWINT);            /* Enable TWI and clear interrupt flag */
        while (!(TWCR & (1<<TWINT)));         /* Wait until TWI finish its current job (read operation) */
        return TWDR;                          /* Return received data */
}
```

Slave Functions:

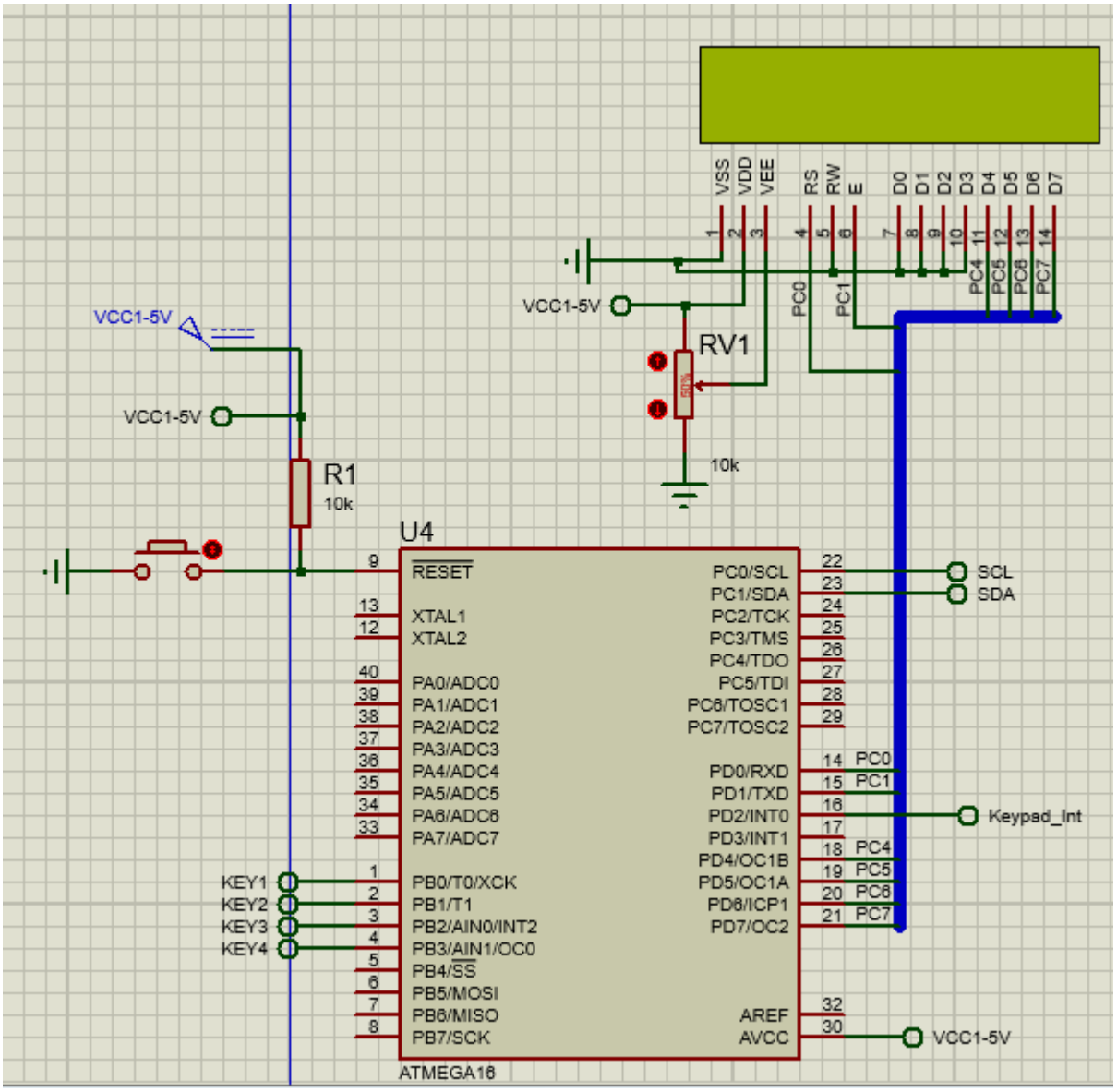
```
#include "I2C_Slave_H_File.h"

void I2C_Slave_Init(uint8_t slave_address)
{
        TWAR = slave_address;                /* Assign address in TWI address register */
        TWCR = (1<<TWEN) | (1<<TWEA) | (1<<TWINT);/* Enable TWI, Enable ack generation, clear TWI interrupt */
}

int8_t I2C_Slave_Listen()
{
        while(1){
        uint8_t status;             /* Declare variable */
        while (!(TWCR & (1<<TWINT)));/* Wait to be addressed */
        status = TWSR & 0xF8;       /* Read TWI status register with masking lower three bits */
        if (status == 0x60 || status == 0x68) /* Check whether own SLA+W received & ack returned (TWEA = 1) */
        return 0;                            /* If yes then return 0 to indicate ack returned */
        if (status == 0xA8 || status == 0xB0) /* Check whether own SLA+R received & ack returned (TWEA = 1) */
        return 1;                            /* If yes then return 1 to indicate ack returned */
        if (status == 0x70 || status == 0x78) /* Check whether general call received & ack returned (TWEA = 1) */
                return 2;                    /* If yes then return 2 to indicate ack returned */
                else
                continue;                    /* Else continue */
        }
}

int8_t I2C_Slave_Transmit(char data)
{
        uint8_t status;
        TWDR = data;                         /* Write data to TWDR to be transmitted */
        TWCR = (1<<TWEN)|(1<<TWINT)|(1<<TWEA);/* Enable TWI and clear interrupt flag */
        while (!(TWCR & (1<<TWINT)));        /* Wait until TWI finish its current job (Write operation) */
        status = TWSR & 0xF8;                /* Read TWI status register with masking lower three bits */
        if (status == 0xA0)                  /* Check whether STOP/REPEATED START received */
        {
                TWCR |= (1<<TWINT);          /* If yes then clear interrupt flag & return -1 */
                return -1;
        }
        if (status == 0xB8)                  /* Check whether data transmitted & ack received */
                return 0;                    /* If yes then return 0 */
        if (status == 0xC0)                  /* Check whether data transmitted & nack received */
        {
                TWCR |= (1<<TWINT);          /* If yes then clear interrupt flag & return -2 */
                return -2;
        }
        if (status == 0xC8)                  /* If last data byte transmitted with ack received TWEA = 0 */
        return -3;                           /* If yes then return -3 */
        else                                 /* else return -4 */
        return -4;
}

char I2C_Slave_Receive()
{
        uint8_t status;                      /* Declare variable */
        TWCR=(1<<TWEN)|(1<<TWEA)|(1<<TWINT);  /* Enable TWI, generation of ack and clear interrupt flag */
        while (!(TWCR & (1<<TWINT)));        /* Wait until TWI finish its current job (read operation) */
        status = TWSR & 0xF8;                /* Read TWI status register with masking lower three bits */
        if (status == 0x80 || status == 0x90) /* Check whether data received & ack returned (TWEA = 1) */
        return TWDR;                         /* If yes then return received data */
        if (status == 0x88 || status == 0x98) /* Check whether data received, nack returned and switched to not
addressed slave mode */
        return TWDR;                         /* If yes then return received data */
        if (status == 0xA0)                  /* Check whether STOP/REPEATED START received */
        {
                TWCR |= (1<<TWINT);          /* If yes then clear interrupt flag & return 0 */
                return -1;
        }
        else
        return -2;                           /* Else return 1 */
}
```

Exercise Task:

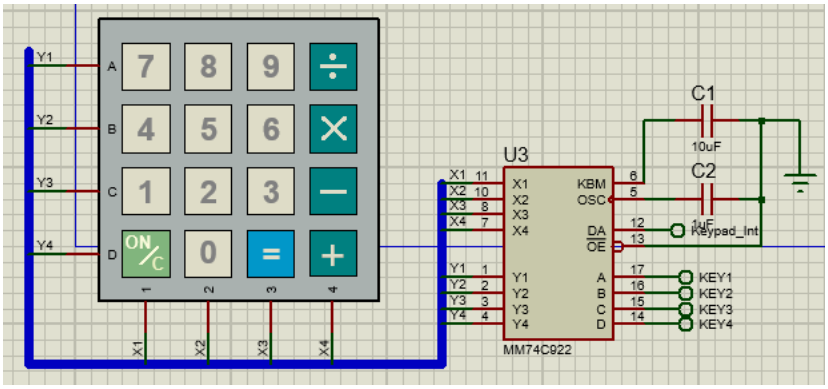
To fully understand the usefulness of I2C, we will implement a 2 way communication channel between a master ATmega16 (MCU1) and a slave ATmega32 (MCU2). The master will feature a keypad interfaced using a MM74C922. A user will be able to send a command message to MCU 2 by typing it on the keypad and pressing the ON/C button to send. Based on the command message, MCU1 will receive sensor readings from MCU 2 and display them on an LCD screen.

MCU2 will feature 2 sensors (potentiometers) and an LCD. It will send the value of the potentiometer to MCU1 depending on the command received and will display the value of any MCU 1 command sent via I2C on the LCD display.

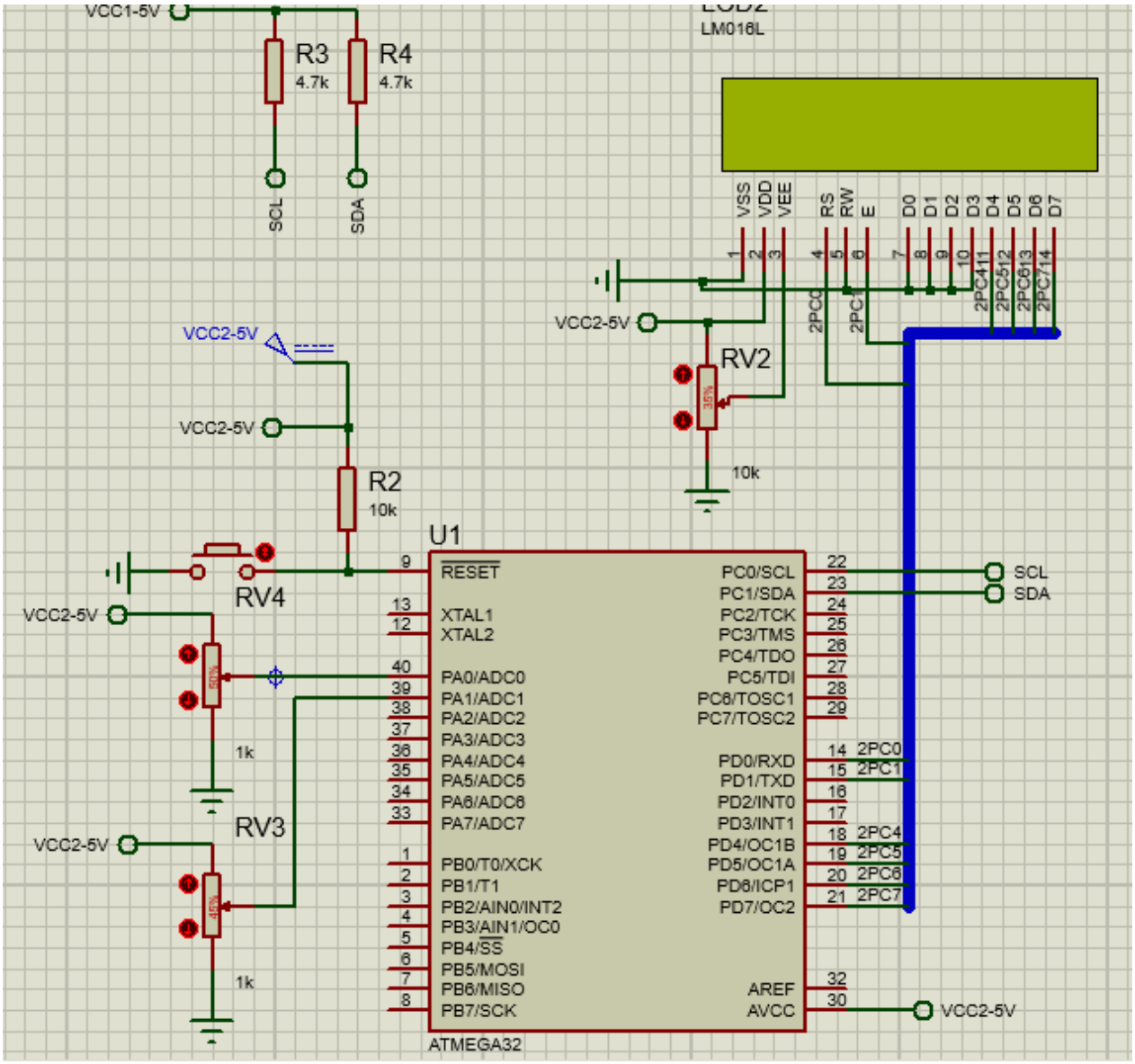
Circuit Implementation:



Master Device



Keypad circuit



Slave Device

Code Section:

```c
/*
 * I2C_MCU1_Atmega16.c
 */

#define F_CPU 8000000UL                         /* Define CPU clock Frequency 8MHz */
#include <avr/io.h>                             /* Include AVR std. library file */
#include <util/delay.h>                         /* Include inbuilt defined Delay header file */
#include <stdio.h>                              /* Include standard I/O header file */
#include <stdlib.h>                             /* Include string header file */
#include <avr/interrupt.h>
#include "I2C_Master_H_file.h"
#include "LCD_PAD.h"

#define Slave_Write_Address           0x20
#define Slave_Read_Address            0x21
#define   count                                         10

int keycount = -1;
uint8_t sensor_select = 0;
uint16_t result;

/*A function to send a serial number the slave and read the slave's result*/
void I2C_Read_Sensor(uint8_t data){
          char buffer[10];
          I2C_Start_Wait(Slave_Write_Address);          // Start I2C with SLA+W
          _delay_ms(50);                                // Important time delay for communication
          I2C_Write(data);                              // Send Incrementing count
          _delay_ms(50);                                // Important time delay for communication
          I2C_Repeated_Start(Slave_Read_Address);       // Repeated Start I2C communication with SLA+R
          _delay_ms(5);                                 // Important time delay for communication
          sprintf(buffer, "%d", I2C_Read_Ack());        // Read and send Acknowledge of data
          _delay_ms(50);                                // Important time delay for communication
          result = I2C_Read_Nack();
          sprintf(buffer, "%d", result);                // Read and Not Acknowledge to data
          LCD_String_xy(1, 11, buffer);
          _delay_ms(50);                                // Important time delay for communication
          I2C_Stop();                                   // Stop I2C
}

/*ISR function: run whenever there is a new key press from the MMC74C922*/
ISR(INT0_vect){
          // Read the value from the keypad connected pins
          char value = PINB & (0x0F);
          uint8_t hold = check_Keypad(value);
          if (hold == 'C') {
                    I2C_Read_Sensor(sensor_select);
          }
          else{
                    sensor_select = hold;
                    LCD_String_xy(1, 0, "Sensor(");
                    LCD_Char(sensor_select);
                    LCD_String("):         ");
          }
}

/* Setup function for the Keypad */
void keypad_Init(){
          DDRD |= (0<<PD2);  /* PORTD as input */
          PORTD |= (1<<PD2); /* Activate pull up resistor high */

          /* Interrupt setup */
          GICR = 1<<INT0;             /* Enable INT0*/
          MCUCR = 1<<ISC01 | 1<<ISC00;  /* Trigger INT0 on rising edge */
          sei();                                /* Enable Global Interrupt */
}

int main()
{
          LCD_Init();                           /* Initialize LCD */
          I2C_Init();                           /* Initialize I2C */
          keypad_Init();
          LCD_String_xy(0, 0, "Master:");
          while (1){
                    // You can put your working code here as the I2C communication is done
```

```
                // through the keypad interrupt.
        }
}
```

```
/*
 * I2C_MCU2_Atmega32.c
 */

#define F_CPU 8000000UL                 /* Define CPU clock Frequency e.g. here its 8MHz */
#include <avr/io.h>                     /* Include AVR std. library file */
#include <util/delay.h>                 /* Include inbuilt defined Delay header file */
#include <stdio.h>                      /* Include standard I/O header file */
#include <stdlib.h>                     /* Include string header file */
#include "LCD_PAD.h"                    /* Include LCD header file */
#include "I2C_Slave_H_File.h"           /* Include I2C slave header file */

#define Slave_Address                           0x20
int8_t sensor_select = 0;


int main(void)
{
        int8_t count = 0;
        char buffer[10];
        ADC_Init();
        LCD_Init();
        I2C_Slave_Init(Slave_Address);
        LCD_String_xy(0, 0, "Slave:");

        while (1)
        {
                switch(I2C_Slave_Listen())          /* Check for any SLA+W or SLA+R */
                {
                        case 0:{
                                do{
                                        count = I2C_Slave_Receive();/* Receive data byte*/
                                        if (count != -1){
                                                sensor_select = count;
                                                sprintf(buffer, "%d", count);
                                                LCD_String_xy(0, 6, "S/No:");
                                                LCD_String(buffer);
                                        }
                                } while (count != -1);      /* Receive until STOP/REPEATED START received */
                                count = 0;
                                break;
                        }
                        case 1:{
                                        int8_t Ack_status;
                                        LCD_String_xy(1, 0, "Sending:          ");
                                        do{
                                                uint8_t result = 0;
                                                if (sensor_select == 49){   /* If byte is '1'= 49*/
                                                        result = ADC_Read(0)/4;
                                                        Ack_status = I2C_Slave_Transmit(result);/* Send data
byte */
                                                        sprintf(buffer, "%d",result);
                                                        LCD_String_xy(1, 9, buffer);
                                                }
                                                else if (sensor_select == 50){/* If byte is '2'= 50*/
                                                        result = ADC_Read(1)/4;
                                                        Ack_status = I2C_Slave_Transmit(result);/* Send data
byte */
                                                        sprintf(buffer, "%d",result);
                                                        LCD_String_xy(1, 9, buffer);
                                                }
                                                else {
                                                        Ack_status = I2C_Slave_Transmit('X'); /* Send data byte
*/
                                                        LCD_String_xy(1, 9, "XXX");
                                                }
                                        }while (Ack_status == 0);   /* Send until Acknowledgment is received */
                                        break;
                                }
                        default:
                        break;
                }
```

```
        }
}
```